

Enterprise Build Plan — fs-hello → Unified Communications Platform

Strategic goal: a Nigerian alternative to **Aircall + RingCentral** (cloud PBX & contact center) and **Intercom** (omnichannel inbox), built voice-first on FreeSWITCH, multi-tenant-ready, launched single-tenant.

This plan turns the locked architecture decisions into a sequenced, milestone-based roadmap. Work is ordered so everything that does **not** depend on the carrier is built and testable first (against the local FreeSWITCH + a softphone), while the carrier trunk is sorted in parallel.

Locked decisions (recap)

Area	Decision
Tenancy	Multi-tenant- ready , launch single-tenant (<code>tenant_id</code> everywhere from day one)
Scale	Design for medium, deploy small (stateless app; DB/Redis/FreeSWITCH as separate services)
Frontend	React + Next.js (App Router)
Calling	WebRTC primary, SIP fallback (browser softphone + external softphones both work)
Hosting	Existing VPS now → separated infrastructure later
Scope	Voice-first, omnichannel foundation
Core platform	Cloud PBX (Extensions, IVR, Queues, Ring Groups, Call Routing, SIP Registration) + Contact Center + Omnichannel Inbox

Recommended technology stack

I implement all of this; these are the tools I'd standardize on. Flag any you want changed.

Layer	Choice	Why
Frontend	Next.js (App Router) + TypeScript , Tailwind + a component kit (shadcn/ui)	Matches the reference platform; fast to build enterprise UIs
Realtime UI	WebSocket (Socket.IO)	Live agent/queue/campaign dashboards
Softphone	SIP.js over FreeSWITCH WSS (<code>mod_sofia</code> WS)	Standards-based WebRTC; SIP fallback comes free
API	NestJS (TypeScript)	Structured, modular, testable — the enterprise- grade replacement for the current single-file Express app
ORM / DB	Prisma + PostgreSQL	Type-safe schema, migrations, multi-tenant scoping
Cache / realtime state / jobs	Redis + BullMQ	Live state (agents, queues, dialer pacing) + background jobs
Telephony	FreeSWITCH + <code>mod_callcenter</code> , <code>mod_avmd</code> , <code>mod_verto</code> /WSS	ACD, answer-machine detection, WebRTC
ESL	One persistent event-socket service	Replaces today's per-call connections; powers realtime + scale
Storage	S3-compatible (MinIO now, S3 later)	Call recordings
Deploy	Docker for stateless app; FreeSWITCH on its own host	Reproducible; media engine isolated

Sub-decision for you: the API rewrite to **NestJS** is my recommendation for long-term maintainability. If you'd rather minimize the rewrite, I can keep a layered **Express/Fastify** structure instead. Everything else is unaffected.

Phase 0 — Foundations (*carrier-independent — start immediately*)

The platform skeleton. Nothing here needs the carrier or any paid account.

- Restructure repo into a monorepo: `apps/web` (Next.js), `apps/api` (NestJS), `packages/shared` .
- **PostgreSQL schema** with `tenant_id` on every table; Prisma migrations.
- **Migrate** existing `data/*.json` into Postgres.
- **Auth**: login/logout, password hashing, sessions/JWT, password reset.
- **RBAC enforced server-side** — your existing Manager-Role permission model becomes real authorization on every endpoint (today it's cosmetic).
- **Persistent ESL service** + **Redis** for live state; replace per-call connections.

- Secrets/config management; CI (lint, typecheck, tests); dev environment.

You provide: nothing required. (Optional: SSH access to the VPS if you want me to deploy as I go.)

Done when: you can log in, the app reads/writes Postgres, permissions are enforced, and the API holds one live FreeSWITCH connection. **Size: L**

Phase 1 — Cloud PBX core (*the foundation everything else rides on*)

This is the RingCentral/Aircall "phone system" layer. Mostly testable locally with softphones.

- **Extensions & SIP registration** — manage users/devices; generate FreeSWITCH directory XML + `reloadxml`.
- **SIP Device** management wired to the directory (strong passwords — fixes the `1234` /attack risk from your session log).
- **Trunk / Gateway** management — configure Sofia gateways from the UI.
- **Call Routing / Dialplan engine** — DID/number → destination, generated as FreeSWITCH XML.
- **IVR builder** — visual menu → dialplan (`mod_ivr`).
- **Ring Groups** — simultaneous/sequential ringing.
- **Queues (ACD)** — install + configure `mod_callcenter` ; tiers/agents/strategies.
- **Time Conditions** — business hours / holiday routing.

You provide: (*parallel track, not blocking*) begin the carrier conversation for a trunk with **outbound enabled + DIDs** (the #1 production blocker from your logs). **Done when:** an internal call flows DID → IVR → queue/ring-group → ringing extension, all configured from the UI. **Size: XL**

Phase 2 — WebRTC softphone + Agent workspace

Calls live in the browser — the defining contact-center experience.

- **FreeSWITCH WSS + TLS** (`mod_verto` /SIP-over-WS).
- **In-browser softphone** (SIP.js): register, answer, hangup, hold, mute, transfer, DTMF.
- **Agent workspace:** presence/status (Available / Break / ACW), screen-pop on inbound, disposition capture.
- **Supervisor tools:** live **Listen / Whisper / Barge** (ESL commands already prototyped) wired into the UI.
- **SIP fallback** confirmed (external softphones keep working for poor-network agents).

You provide: TLS via your domain's DNS (I automate Let's Encrypt). **Done when:** an agent logs in, sets Available, and answers a queued call entirely in the browser; a supervisor can monitor it. **Size: L**

Phase 3 — Contact Center & Dialer (*depth*)

Turns voice-broadcast into a real contact center where customers reach live agents.

- **Outbound campaigns that bridge answered calls to an available agent** (via ACD) — not just play audio.
- **Predictive/Progressive pacing** with **AMD** (`mod_avmd`) and **abandon-rate control**.
- **Inbound campaigns** → real routing into queues/agents.
- **Blended** inbound+outbound.
- **Leads, Lead Groups, Custom Fields, DNC, Dispositions, Callbacks/Follow-ups** on Postgres.
- **Real-time dashboards** (agents, queues, campaigns) over WebSocket.
- **Recordings** → S3/MinIO with retrieval + playback.
- **Reports** wired to real DB data (CDR, agent performance, campaign, SLA).

You provide: object storage (MinIO on the VPS now, S3 later); the carrier trunk for *real* outbound dialing tests. **Done when:** a campaign dials, detects humans vs machines, connects live answers to agents, records the call, and totals roll into live dashboards + reports. **Size: XL**

Phase 4 — Omnichannel Inbox foundation (*the "Intercom" layer*)

Build the channel-agnostic core now; light up channels as accounts arrive.

- **Unified conversation model** (voice, SMS, WhatsApp, email share one inbox/thread structure).
- **Inbox UI** (Intercom-style): conversations, assignment/routing, internal notes, canned responses, contact timeline.
- **Channel adapters:** SMS first, then WhatsApp, then email — each behind a common interface.
- **Webform / live-chat** → conversation.

You provide (per channel, only when you want it live): SMS gateway (Termii/Twilio), **WhatsApp Business API** (Meta approval — start early, it's slow), SMTP/ESP for email. **Done when:** an inbound SMS and a voice call appear as conversations in one inbox, assignable to agents. **Size: L (foundation) + M per channel**

Phase 5 — Multi-tenant SaaS activation + productization

Flip on the SaaS business (the tenancy machinery from Phase 0 makes this incremental).

- Tenant **onboarding & provisioning**; isolation hardening + per-tenant limits.
- **Billing** (Paystack/Flutterwave): plans, usage metering, invoices.
- **Super-admin console**; per-tenant **white-label/branding**.
- Number-provisioning workflow.

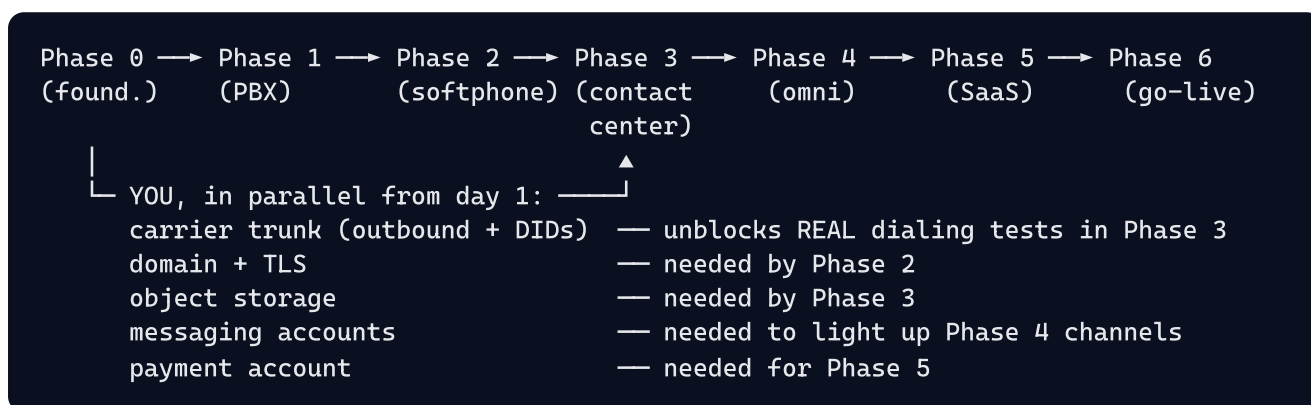
You provide: payment/merchant account; pricing/plan definitions; branding assets. **Done when:** a second tenant can be onboarded, isolated, and billed end to end. **Size: L**

Phase 6 — Hardening & go-live

- **Independent security review / pen-test** (third party) + load testing.
- **Observability**: metrics, centralized logs, alerting; **backups & DR**.
- **Separated infrastructure**: move FreeSWITCH to its own VM, DB to managed/dedicated.
- Runbooks & operational docs.

You provide: the pen-test vendor; cloud accounts for separated infra. **Done when**: the system passes security + load testing and runs on role-separated infrastructure. **Size**: M–L

Critical path & parallelism



Key point: Phases 0–2 and most of 3 can be built and demoed on the **local FreeSWITCH with softphones** — so engineering is never idle waiting on the carrier. The carrier only blocks *real outbound calling tests*, which is a Phase 3 concern.

What only you can provide — consolidated, by when

Needed by	Item (you only)
Phase 2	Domain + DNS control (for TLS)
Phase 3	Carrier trunk with outbound enabled + DIDs (start now — long lead time); object storage
Phase 3+	Telecom regulatory compliance (NCC, caller-ID, consent/DNC)
Phase 4	SMS / WhatsApp / Email provider accounts
Phase 5	Payment/merchant account; pricing; branding
Phase 6	Pen-test vendor; cloud accounts for separated infra
Ongoing	Production server budget; secrets/credentials for the above

Recommended immediate start

Begin **Phase 0** now — it's pure engineering, needs nothing from you, and every later phase depends on it. In parallel, please open the **carrier conversation** (outbound-enabled trunk + DIDs), since that's the longest external lead time and the one true blocker for live calling.

Say the word and I'll start Phase 0: monorepo + Postgres schema + auth/RBAC + the persistent FreeSWITCH service.